Programming Quantum Computers (Multi-Qubit Primitive Operations)

(**Subtrack of** Quantum Computing: An App-Oriented Approach)

Moez A. AbdelGawad

moez@{cs.rice.edu, alexu.edu.eg, srtacity.sci.eg}

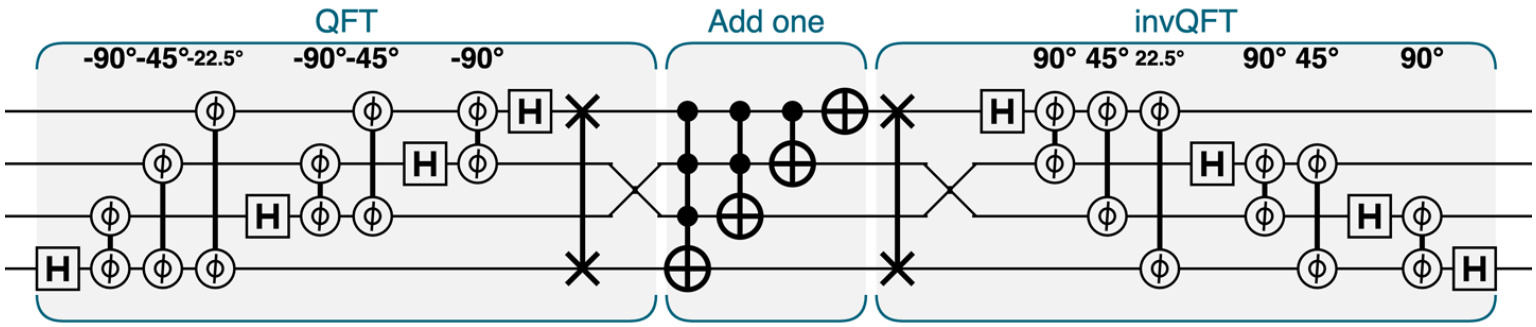
Sat., Nov. 2nd, 2019

Quantum Computers are Real

- What are they <u>useful</u> for?
  - Let's discover, by programming them!

- A hands-on approach to programming QCs/QPUs.
  - By doing; i.e., by writing code & building programs.
  - Using simulators, since real QCs are harder-to-access (so far).

- Goals: Read, understand, write, and *debug* quantum programs.
  - Ones like the following.



Copyright © 2019-24 Moez A. AbdelGawad

Lecture Outline

- Multi-Qubit Operations
  - Entangled Quantum States.
  - Operator Pairs of Single-Qubit Ops.
  - Controlled Operations.
    - CNOT, CPHASE, …, etc.
- Teleportation.
  - IBM QX (Quantum eXperience).

Copyright © 2019-24 Moez A. AbdelGawad

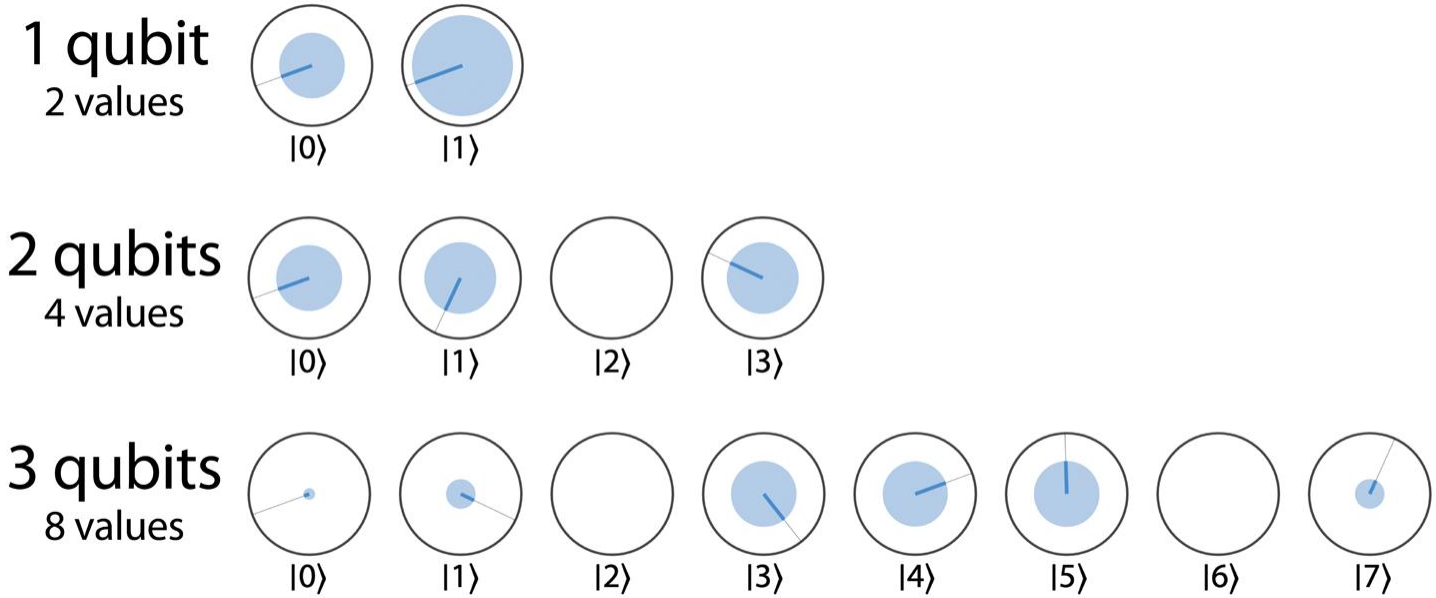
Section I

MULTIPLE QUBITS AND MULTI-QUBIT OPERATIONS

Copyright © 2019-24 Moez A. AbdelGawad

Entanglement

- A very special kind of *qubit interaction*.
  - Interaction implies interdependence, which implies that an amplitude (a magnitude and a phase) is associated with each possible *value*.
- Convenient circle notation for *N* qubits: Separate circle for each of the 2^N different values.
  - Magnitude tells probability the value will be observed when *all* qubits are read ($\Sigma = 1$).

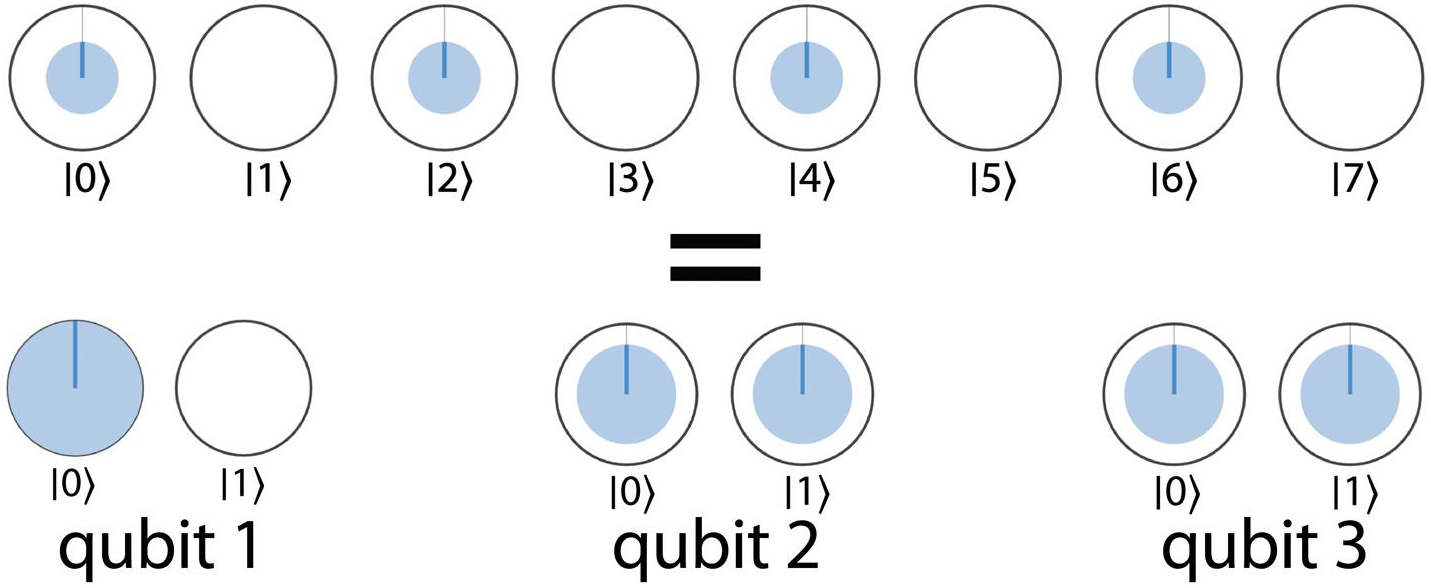


© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

Entanglement

- Some joint multi-qubit states can be expressed using multiple single-qubit states.



© Programming Quantum Computers: O'Reilly Media

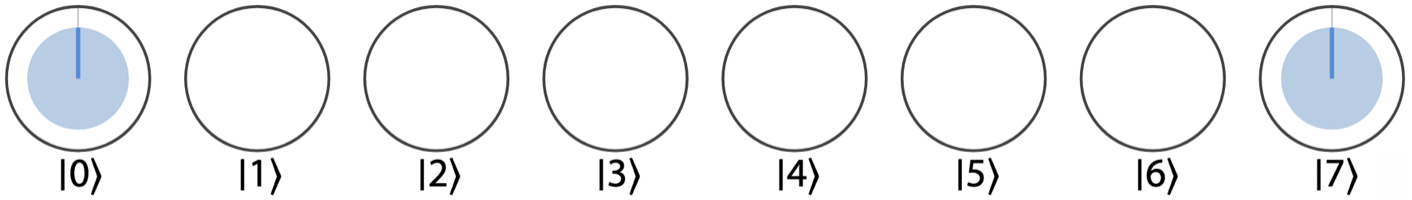
Copyright © 2019-24 Moez A. AbdelGawad

Entanglement

- Label qubits using their contributed value (hex).

```
qc.reset(3)
qc.write(0)   // Hands-on: Try different values in {0, 1}
qc.had(0x2) // Hands-on: Try different qubits in
qc.had(0x4) //  {0x1, 0x2, 0x4}
```

- But some joint states can't be separated… Entanglement!

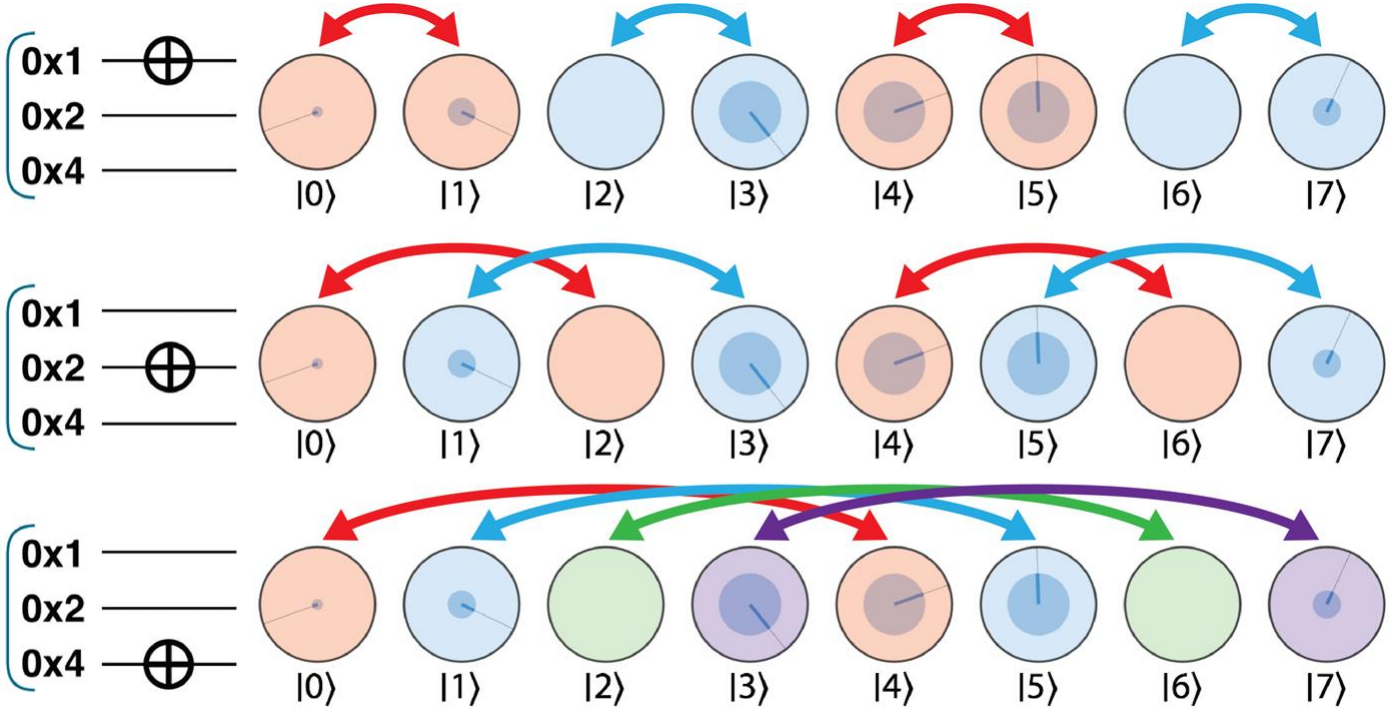


  - Equal superposition of $|000\rangle = |0\rangle$ and $|111\rangle = |7\rangle$ (no others).
  - Impossible to separate. Try it! (Some link between qubits).
  - To produce this state *multi-qubit* operations are needed.

Copyright © 2019-24 Moez A. AbdelGawad

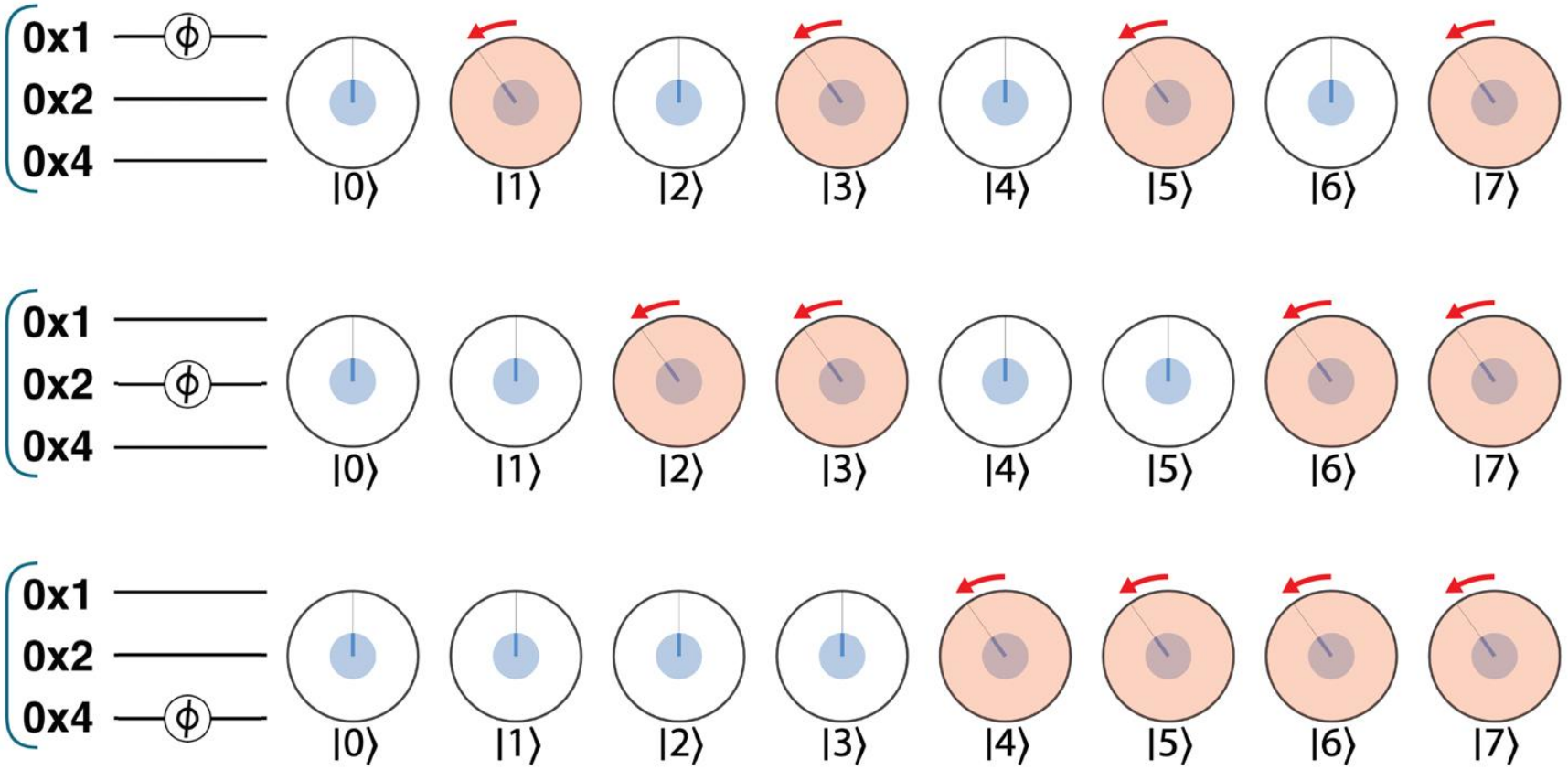
Operator Pairs (⊕)

- Effect of single-qubit ops (e.g., NOT) on joint state.



Copyright © 2019-24 Moez A. AbdelGawad

Operator Pairs (ϕ)



© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

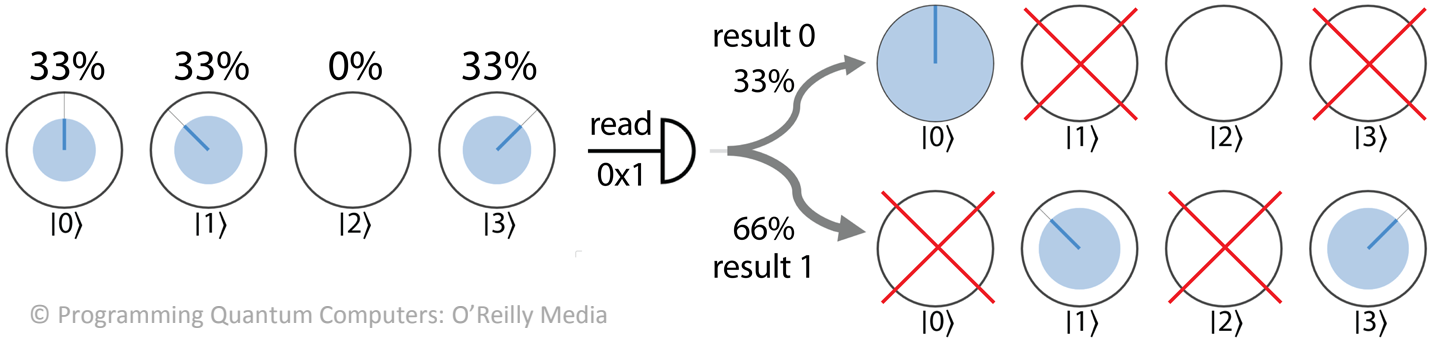
Operator Pairs (H)

- Using operator pairs (of H), explain how state on Slide 6 is obtained from code on Slide 7?
  - Step-by-step.
  - Exercise: Do it on paper, or whiteboard.

Copyright © 2019-24 Moez A. AbdelGawad

Renormalization

- Effect of READing a *single* qubit.
  - Probabilities of possible values change, so as to sum to 1.0 (amplitudes *renormalize*).
  - Circles disagreeing with result eliminated (p=0.0).



© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

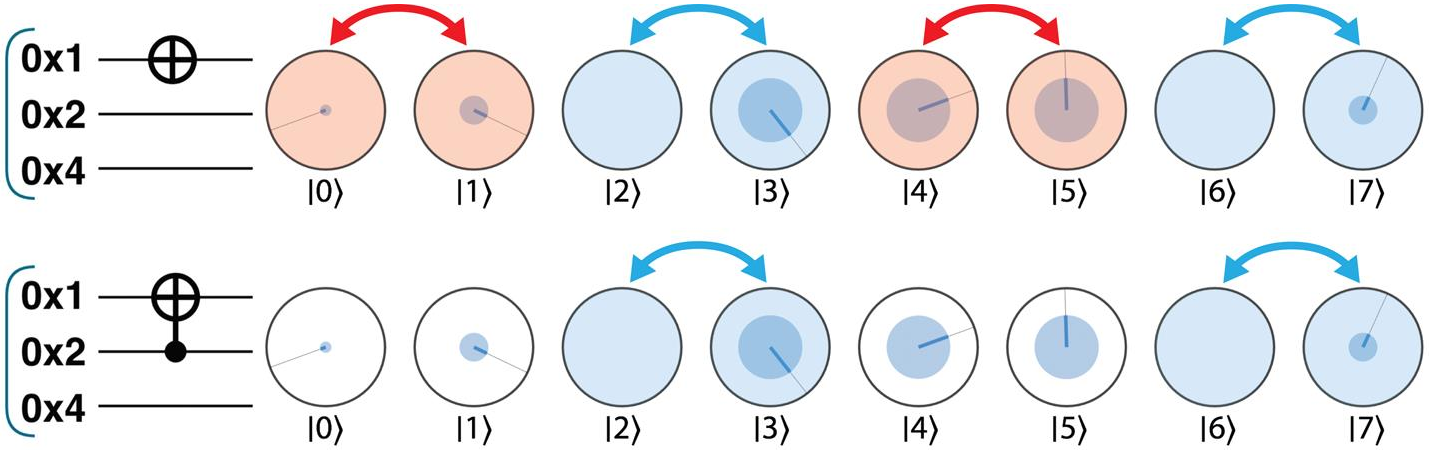
Controlled Operations

- Ops that *require* more than one qubit.
  - Some *control* qubits are used as conditions for performing an operation on a *target* qubit.
    - Similar to “if” in conventional programming.
  - Controlled Not, Controlled Phase, Controlled Hadamard, … etc.

Copyright © 2019-24 Moez A. AbdelGawad

CNOT vs. NOT

- CNOT (‘Controlled NOT’) selectively applies NOT.



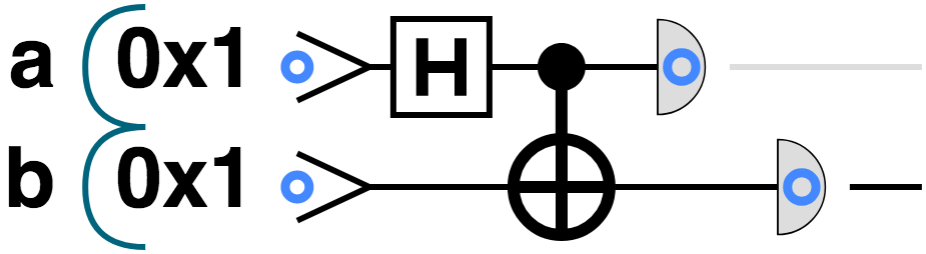
© Programming Quantum Computers: O’Reilly Media

  - Only values with *second* qubit set are affected by NOT.
  - Like NOT, operation CNOT is its own inverse (clear from diagram).

Copyright © 2019-24 Moez A. AbdelGawad

Entanglement Ops

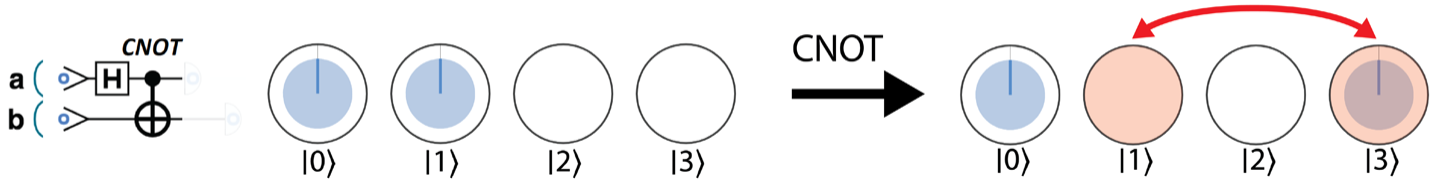
- Unanswered so far:
  - How to cause entanglement (‘inseparable states’)?
  - What happens when control qubits of controlled ops are in (perfect) superposition (“undecided”)?



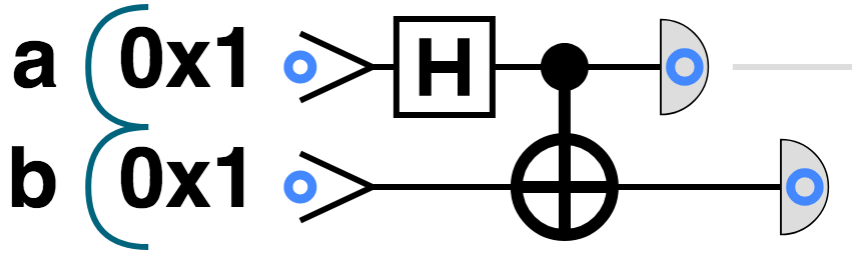
© Programming Quantum Computers: O’Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

Entanglement Ops



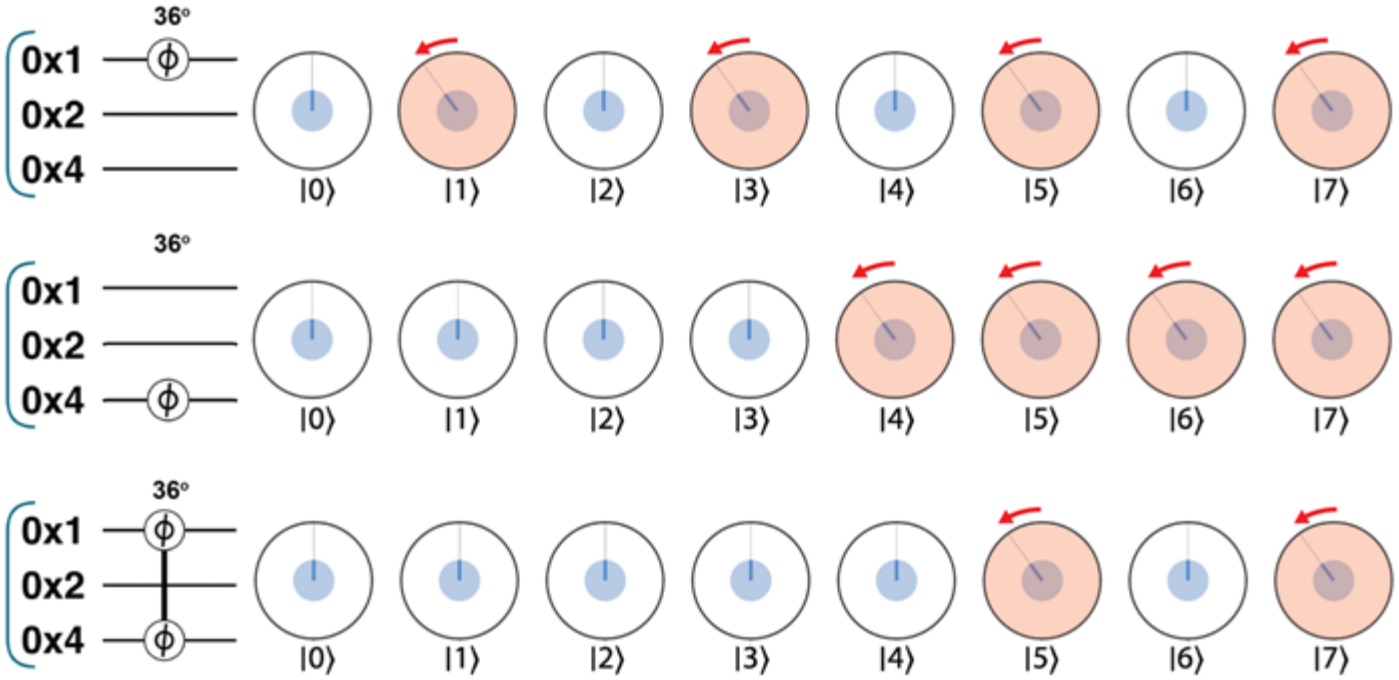
© Programming Quantum Computers: O'Reilly Media



- Although outcomes of the two reads are random, the two outcomes *always agree*.
  - No common cause, no communication, but *entanglement*!
  - Qubits are entangled ('correlated') no matter how far apart they are.
  - Basis for *secure* shared randomness (i.e., secure communication/Internet).

Copyright © 2019-24 Moez A. AbdelGawad

Controlled PHASE (CPHASE)

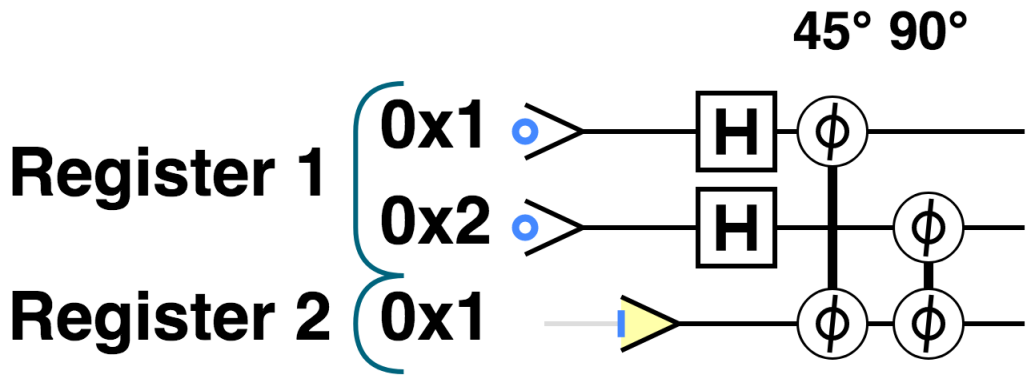


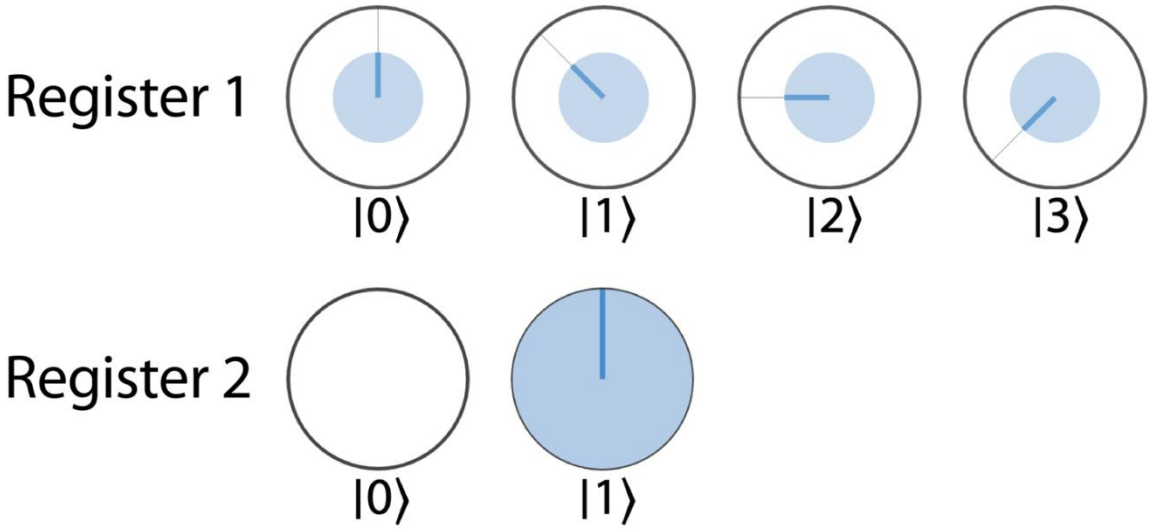
(Figure 3-16 in textbook is missing shading on first row, for value |1⟩)

- For CPHASE, control/target qubit distinction is irrelevant (symmetry).
- Condition cuts the number of rotated circles in half (conjunction/intersection).

Copyright © 2019-24 Moez A. AbdelGawad

Phase Kickback





© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

Phase Kickback

- Hands-on: Play with it. Useful in many applications.

```
qc.reset(3);
// Create two registers
var reg1 = qint.new(2, 'Register 1');
var reg2 = qint.new(1, 'Register 2');
reg1.write(0);
reg2.write(1);   //   VIP

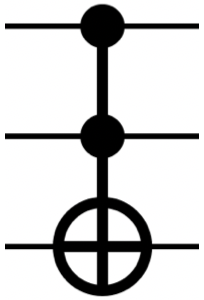
// Place the first register in superposition
reg1.had();

// Perform phase rotations on second register,
// conditioned on qubits from the first
qc.phase(45, 0x4, 0x1);
qc.phase(90, 0x4, 0x2);
```


Copyright © 2019-24 Moez A. AbdelGawad

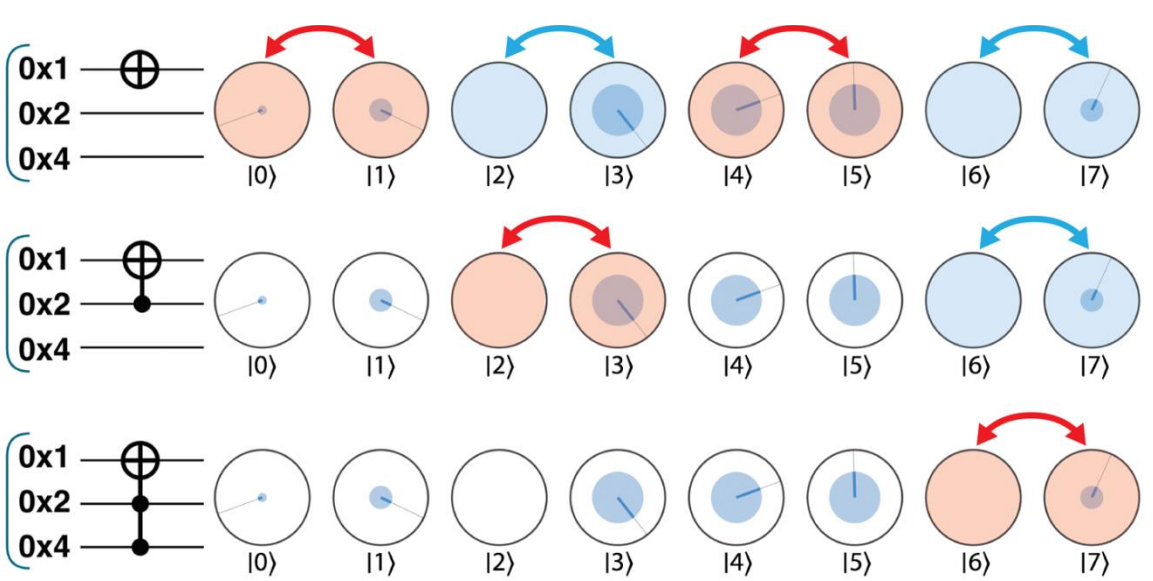
Toffoli Gate (CCNOT)

- ‘More selective by more conditioning’.
  - “if A and B then Op(C)”.
  - CCNOT = NOT conditioned on *two* qubits.



Copyright © 2019-24 Moez A. AbdelGawad

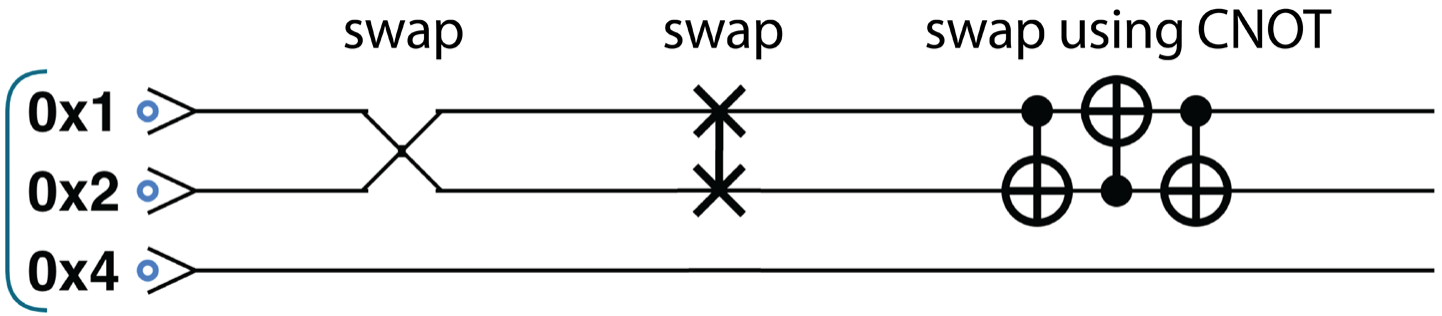
Toffoli Gate (CCNOT)



Exercise: What if 0x2 or 0x4 is the target qubit?

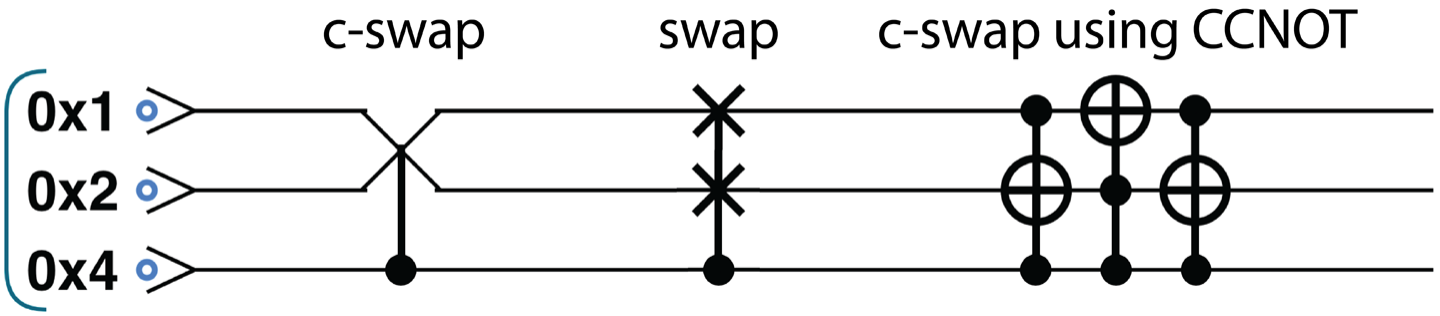
Copyright © 2019-24 Moez A. AbdelGawad

CSWAP



(Similar to using XOR for swapping in classical logic)

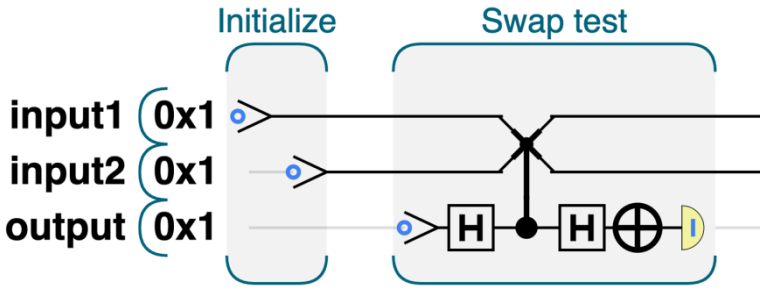
© Programming Quantum Computers: O'Reilly Media



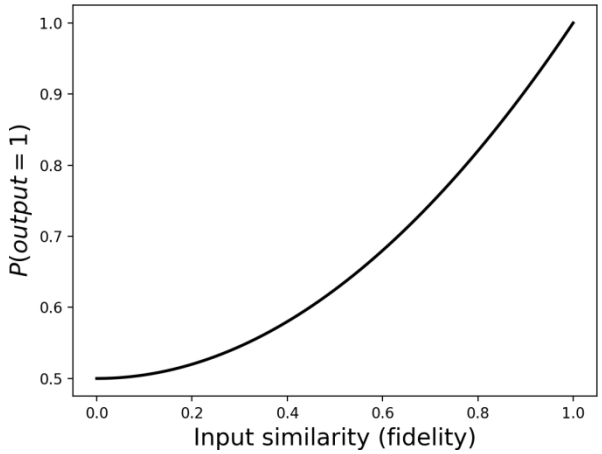
Copyright © 2019-24 Moez A. AbdelGawad

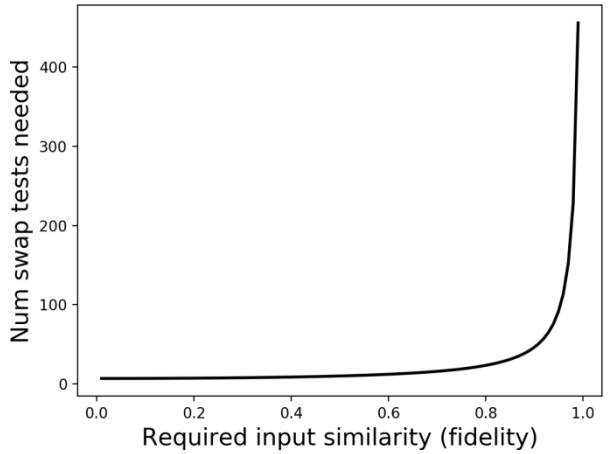
Swap Test

- Similar to equality testing. Useful in QML apps.



© Programming Quantum Computers: O'Reilly Media

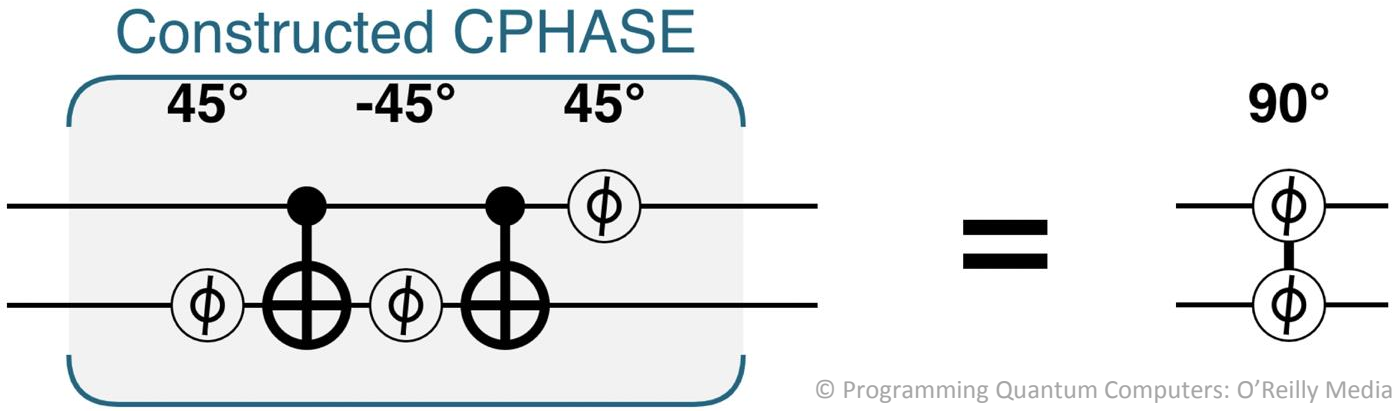




Copyright © 2019-24 Moez A. AbdelGawad

CHAD, CRNOT, …

- Conditioning any operation.
- E.g., constructing CPHASE from CNOT and PHASE.



© Programming Quantum Computers: O'Reilly Media

- For conditioning other operations, procedure slightly more complicated (Ch. 14).

Copyright © 2019-24 Moez A. AbdelGawad

Hands-on:
Remote-Controlled Randomness

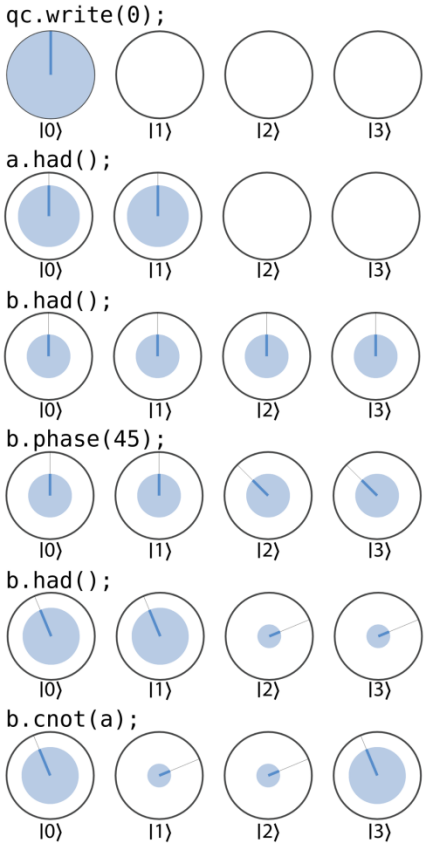
```
qc.reset(2);
var a = qint.new(1, 'a');
var b = qint.new(1, 'b');

qc.write(0);
a.had();
// now prob of a is 50%

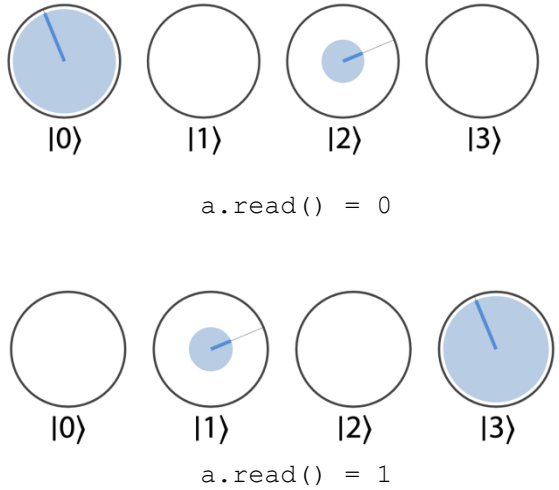
b.had();
b.phase(45);
b.had();
// now prob of b is 15%

b.cnot(a);
// Now, you can read *either*
// qubit and get 50% prob.
// If the result is 0, then
// the prob of the *remaining*
// qubit is 15%, else it's 85%.

var a_result = a.read();
var b_result = b.read();
```



© Programming Quantum Computers: O'Reilly Media



Exercise: Change in code to guarantee *always* getting inverses. (Answer at end of lecture.)

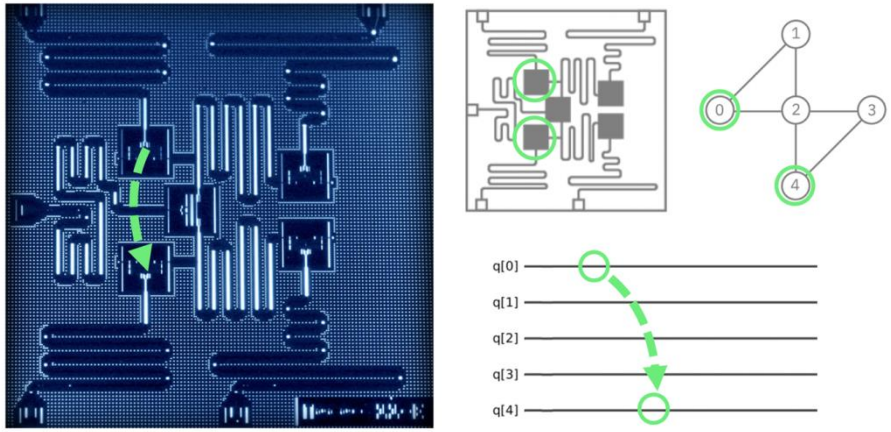
Copyright © 2019-24 Moez A. AbdelGawad

Section II

TELEPORTATION

Copyright © 2019-24 Moez A. AbdelGawad

Quantum Teleportation



- IBM Q Experience (QX, Quantum eXperience).
  - Use IBM's five-qubit real QPU.
  - Can be programmed using OpenQASM code to run on real chip (needs IBM account, more time), or using Python code to run on Qiskit.
  - We'll use OpenQASM to perform teleportation of quantum info (i.e., quantum state—magnitudes and phase) from qubit q[0] to qubit q[4].

Copyright © 2019-24 Moez A. AbdelGawad

Teleportation: OpenQASM on IBM QX (2018)

```
include "qelib1.inc";
qreg q[5];
creg c[5];

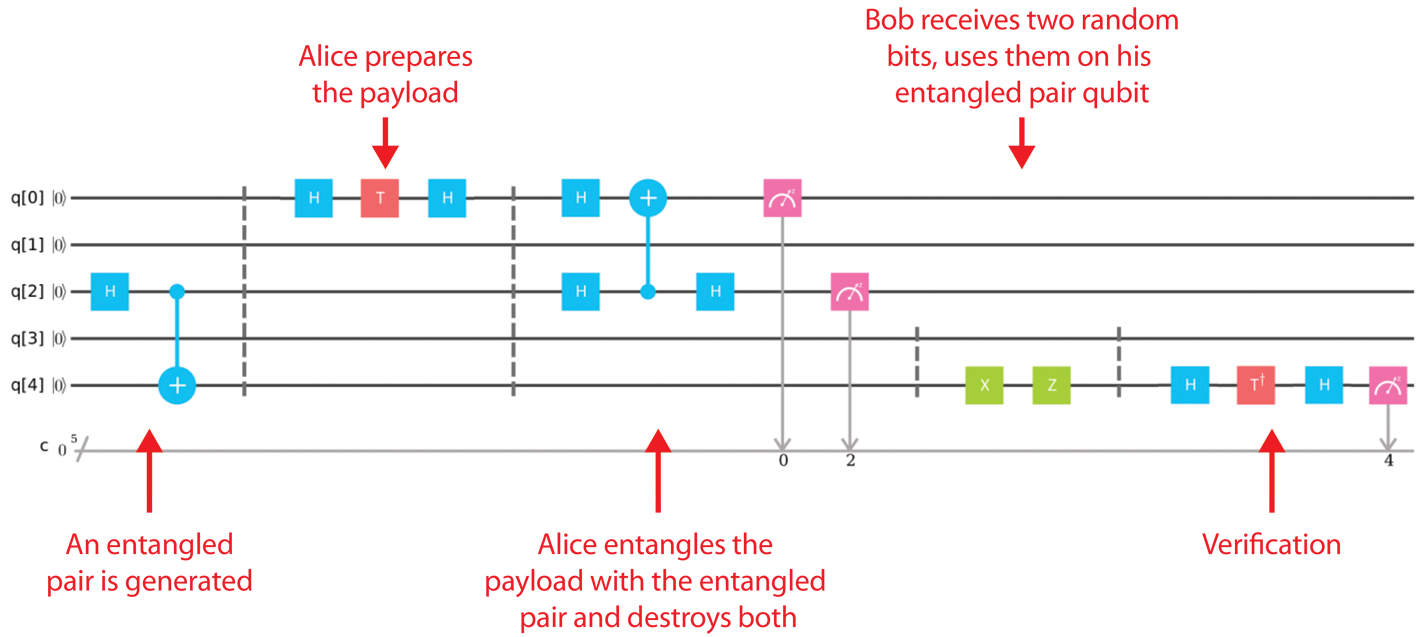
// Step 1: Create entangled
// pair
h q[2];
cx q[2],q[4];
barrier
q[0],q[1],q[2],q[3],q[4];
```

```
// Step 2: Prepare a payload
x q[0];
h q[0];
t q[0];
barrier q[0],q[1],q[2],q[3],q[4];
```

```
// Step 3: Send
h q[0];
h q[2];
cx q[2],q[0];
h q[2];
measure q[0] -> c[0];
measure q[2] -> c[2];
barrier q[3],q[4];
```

```
// Step 4: Receive
x q[4];
z q[4];
barrier q[3],q[4];

// Step 5: Verify
tdg q[4];
h q[4];
x q[4];
measure q[4] -> c[4];
```



© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

Teleportation: OpenQASM on IBM QX (2019)

```
include "qelib1.inc";
qreg q[5];
creg c[5];

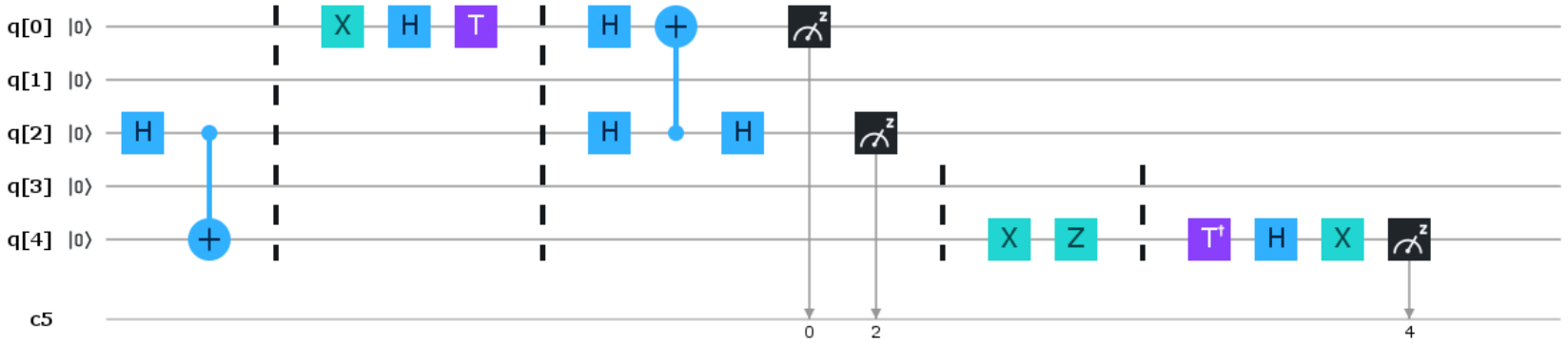
// Step 1: Create entangled
// pair
h q[2];
cx q[2],q[4];
barrier
q[0],q[1],q[2],q[3],q[4];
```

```
// Step 2: Prepare a payload
x q[0];
h q[0];
t q[0];
barrier q[0],q[1],q[2],q[3],q[4];
```

```
// Step 3: Send
h q[0];
h q[2];
cx q[2],q[0];
h q[2];
measure q[0] -> c[0];
measure q[2] -> c[2];
barrier q[3],q[4];
```

```
// Step 4: Receive
x q[4];
z q[4];
barrier q[3],q[4];

// Step 5: Verify
tdg q[4];
h q[4];
x q[4];
measure q[4] -> c[4];
```



Copyright © 2019-24 Moez A. AbdelGawad

Teleportation: Python on Qiskit

```
# Set up the program
alice = QuantumRegister(1)
ep    = QuantumRegister(1)
bob   = QuantumRegister(1)
alice_c = ClassicalRegister(1)
ep_c    = ClassicalRegister(1)
bob_c   = ClassicalRegister(1)
qc = QuantumCircuit(alice, ep,
     bob, alice_c, ep_c, bob_c)

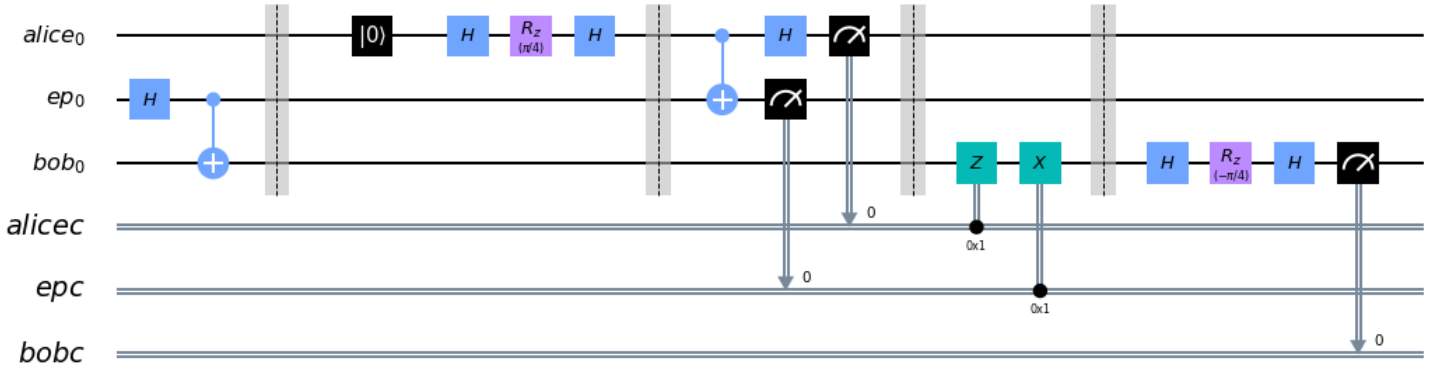
# entangle
qc.h(ep)
qc.cx(ep, bob)
qc.barrier()
```

```
# prep payload
qc.reset(alice)
qc.h(alice)
qc.rz(math.radians(45),
alice)
qc.h(alice)
qc.barrier()
```

```
# send
qc.cx(alice, ep)
qc.h(alice)
qc.measure(alice, alice_c)
qc.measure(ep, ep_c)
qc.barrier()
```

```
# receive
qc.z(bob).c_if(alice_c, 1)
qc.x(bob).c_if(ep_c, 1)
qc.barrier()

# verify
qc.h(bob)
qc.rz(math.radians(-45), bob)
qc.h(bob)
qc.measure(bob, bob_c)
```



Copyright © 2019-24 Moez A. AbdelGawad

Teleportation: JavaScript on QCEngine

```
qc.reset(3);
var alice = qint.new(1);
var ep    = qint.new(1);
var bob   = qint.new(1);
var a1 = 0;
var a2 = 0;

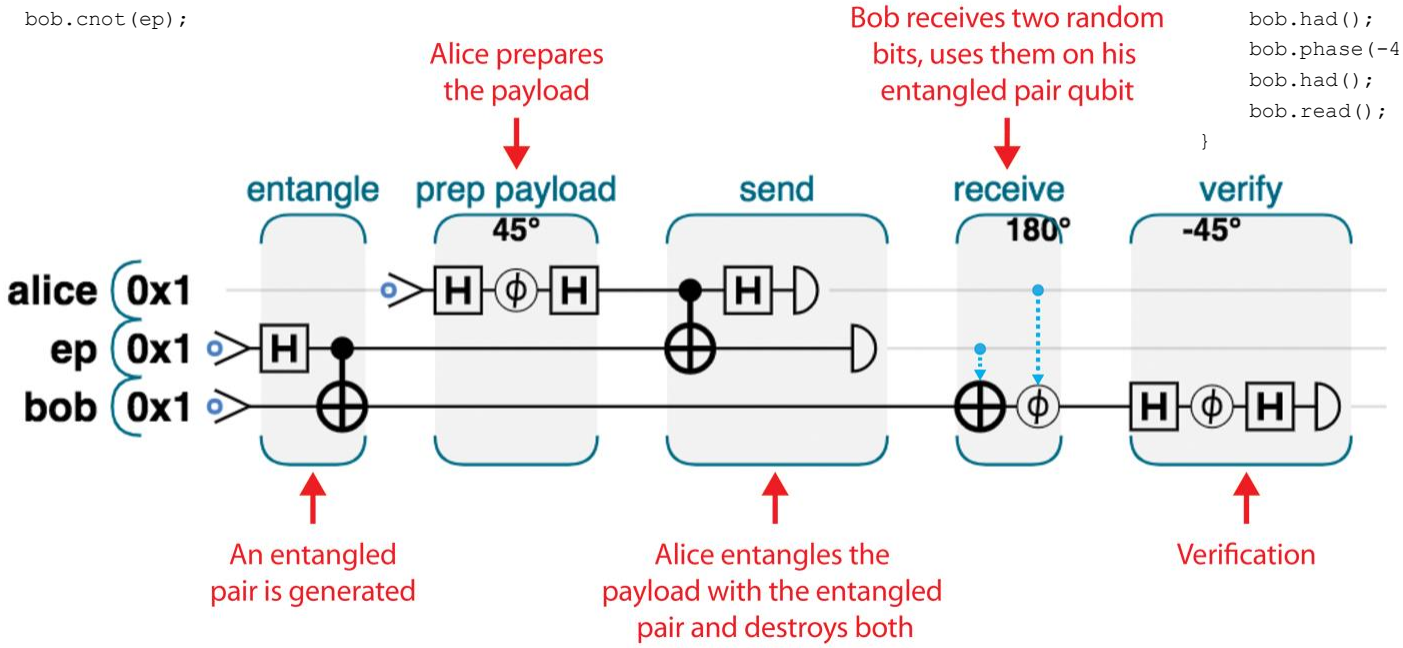
function entangle()
{
    qc.write(0, 2|4);
    ep.had();
    bob.cnot(ep);
}
```

```
function alice_prep()
{
    // Alice prepares her
    // payload to teleport
    alice.write(0);
    alice.had();
    alice.phase(45);
    alice.had();
}
```

```
function alice_send()
{
    // Alice sends payload
    // (destroyed in process)
    ep.cnot(alice);
    alice.had();
    a1 = alice.read();
    a2 = ep.read();
}
```

```
function bob_receive()
{
    // Bob receives payload,
    // using Alice's two bits
    if (a1) bob.phase(180);
    if (a2) bob.not();
}

function bob_verify()
{
    // Verify teleportation
    bob.had();
    bob.phase(-45);
    bob.had();
    bob.read();
}
```



Copyright © 2019-24 Moez A. AbdelGawad

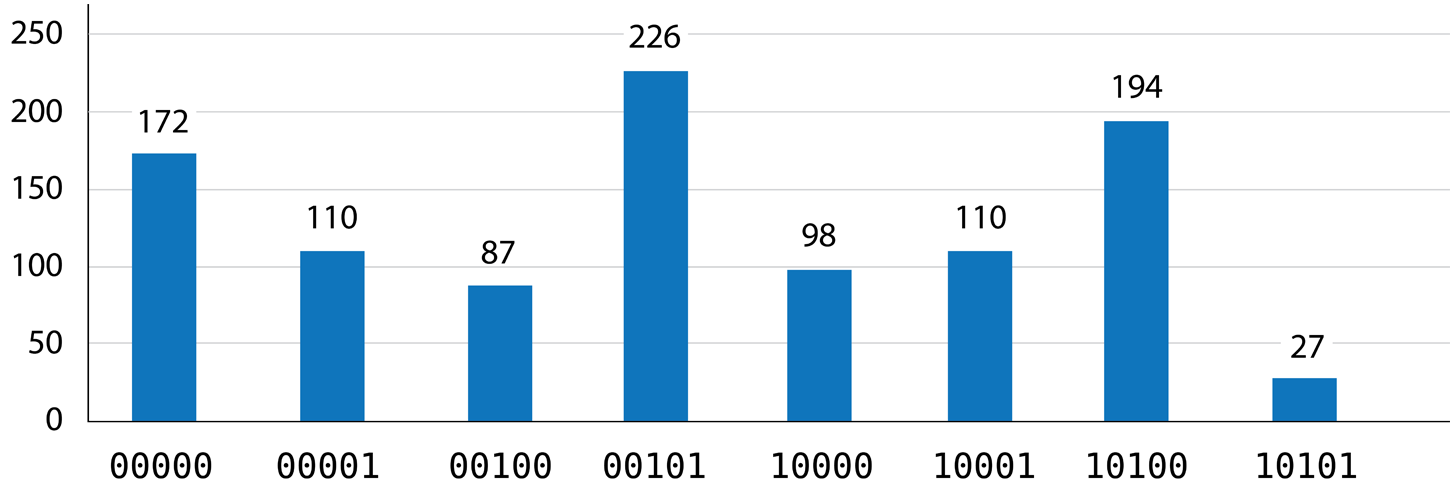
Teleportation:
Q# on Quantum .NET

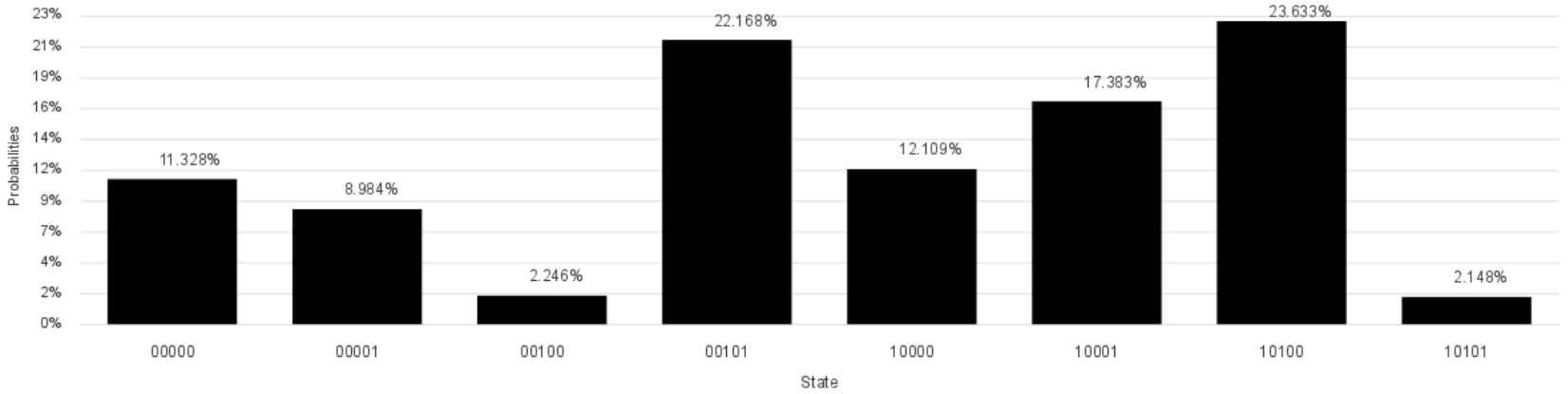
Homework!

- Further: On Cirq, Quirk, …, and other quantum platforms and simulators.

Copyright © 2019-24 Moez A. AbdelGawad

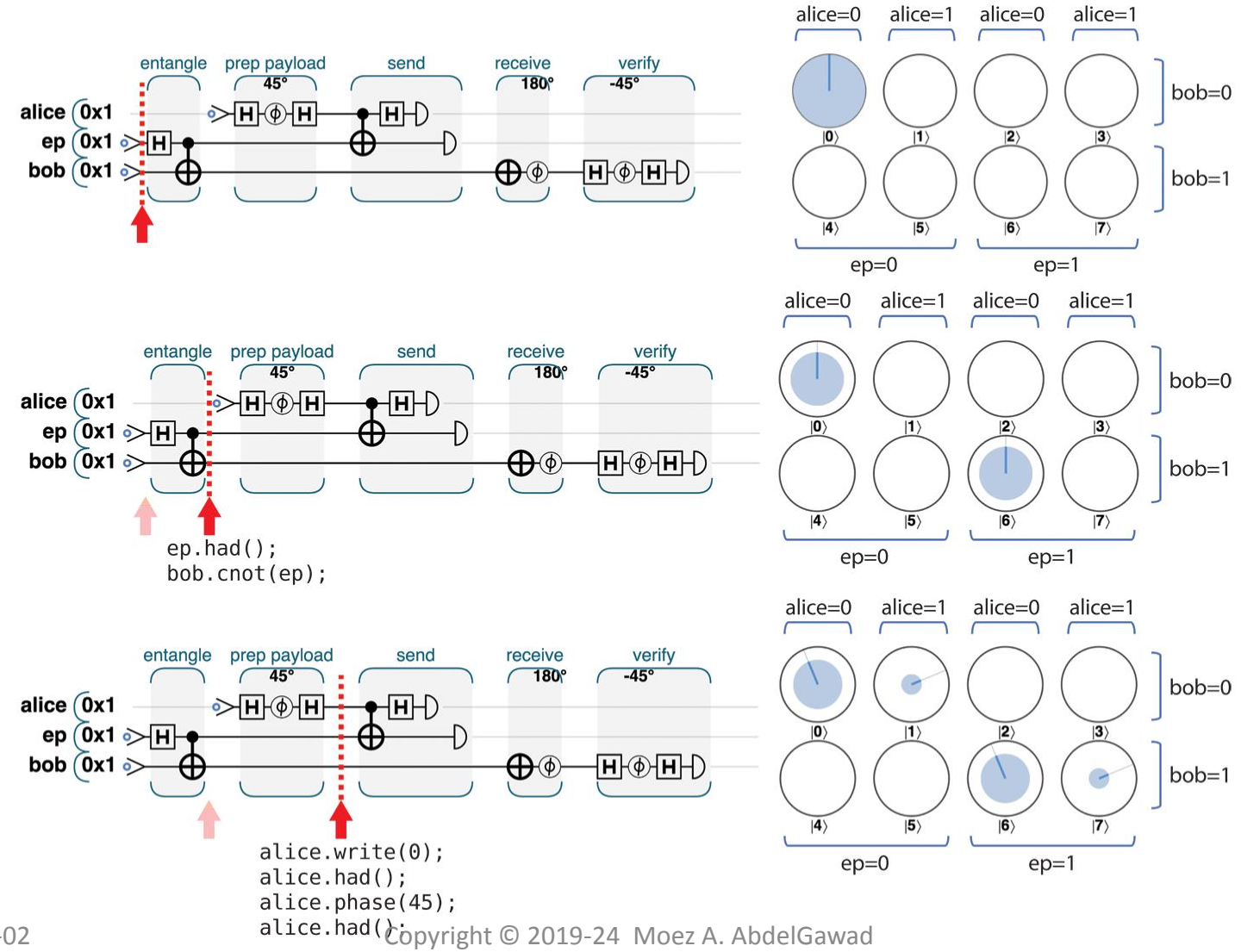
Teleportation: Results on QX. Success??





Copyright © 2019-24 Moez A. AbdelGawad

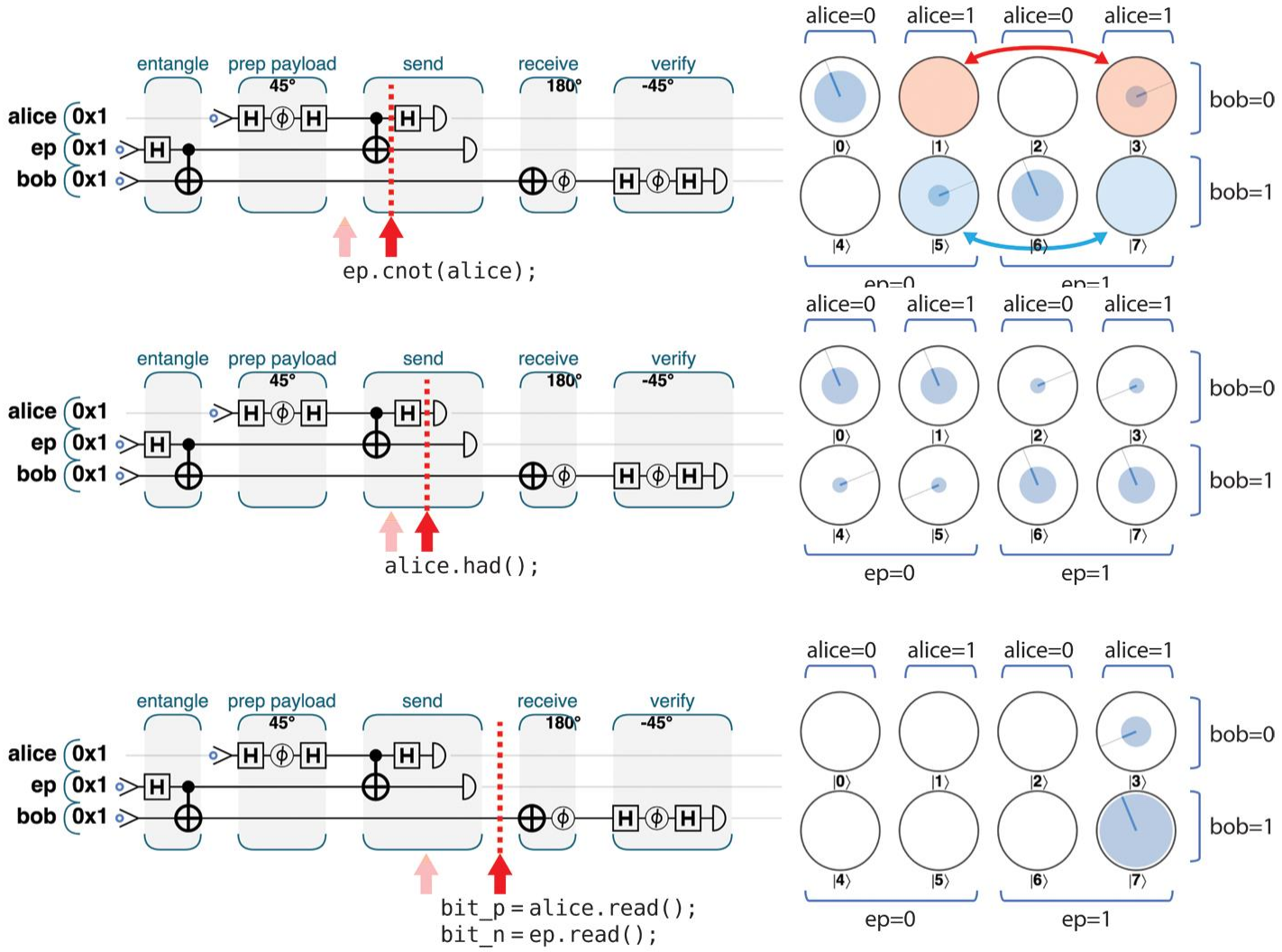
Teleportation: Interpretation



© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

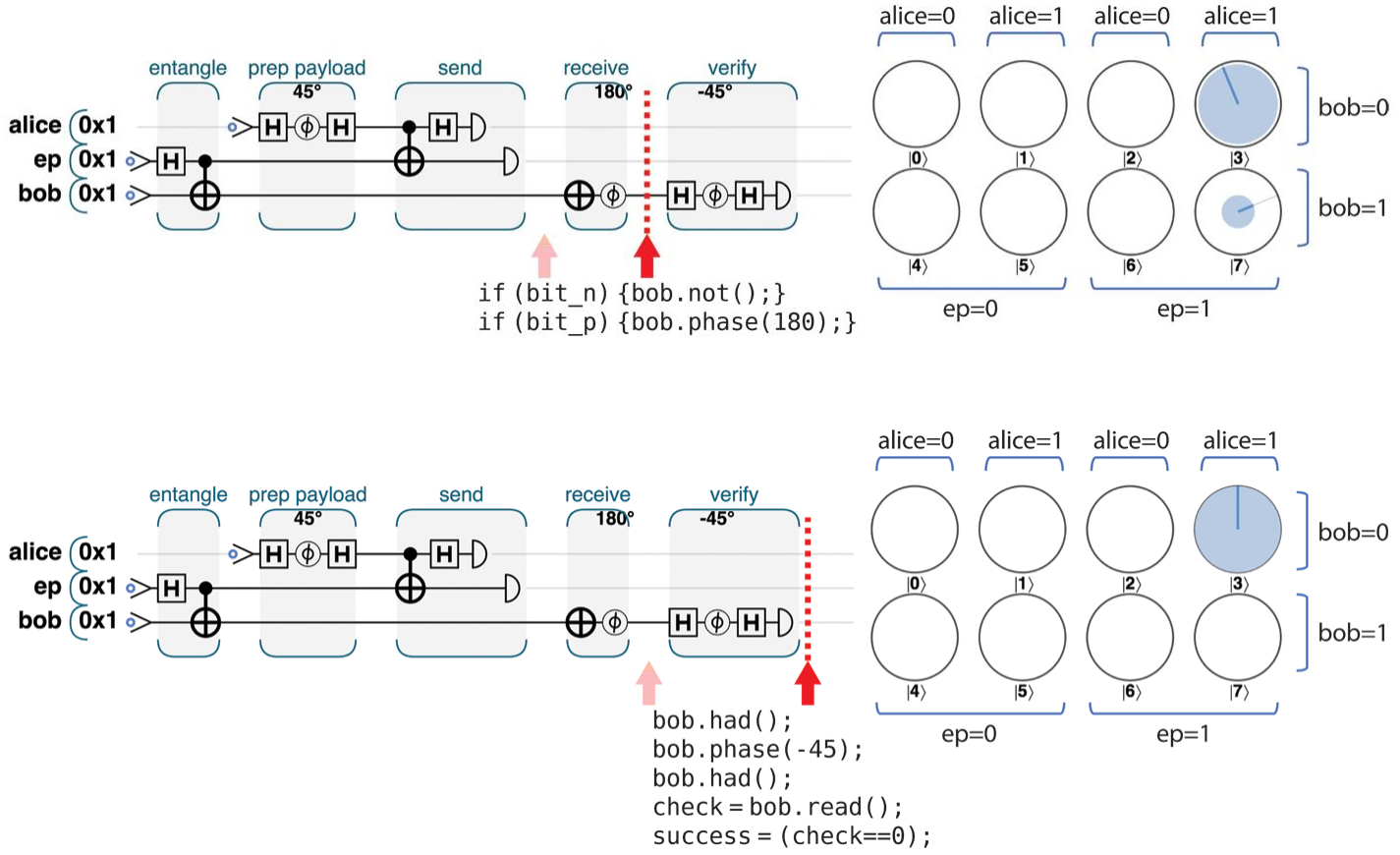
Teleportation: Interpretation



© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

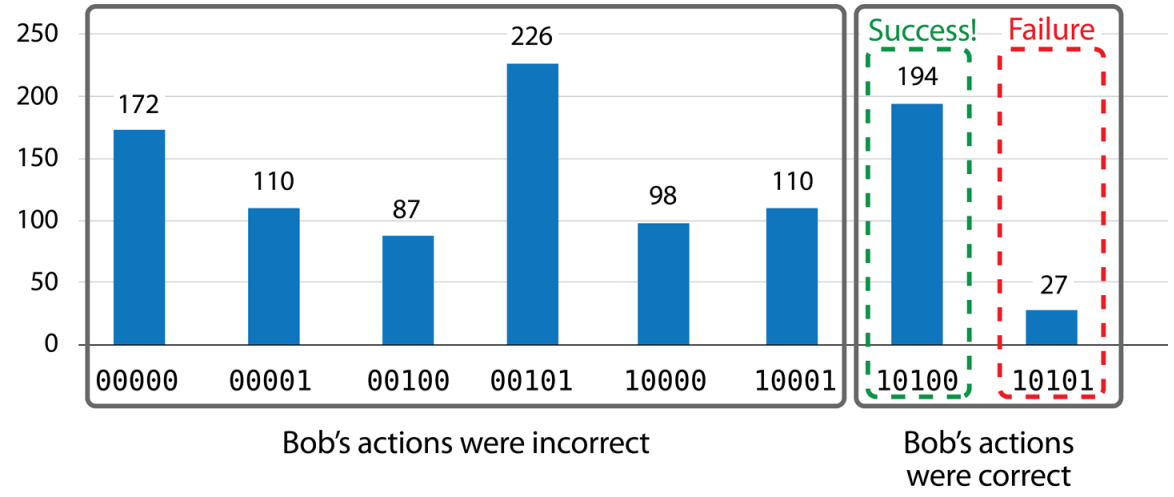
Teleportation: Interpretation

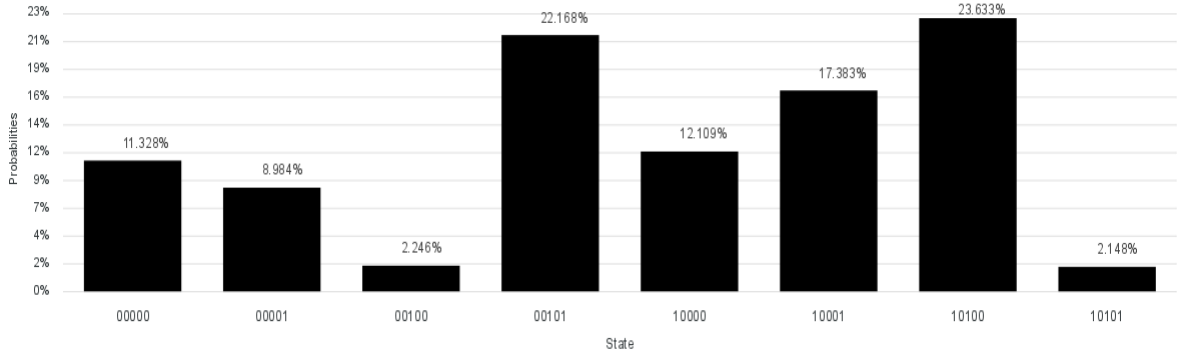


© Programming Quantum Computers: O'Reilly Media

Copyright © 2019-24 Moez A. AbdelGawad

Teleportation:
IBM QX Results Interpretation





(*Feed-forward* isn't supported by IBM QX. Use *post-selection* instead.)

Copyright © 2019-24 Moez A. AbdelGawad

Discussion

Q & A

Copyright © 2019-24 Moez A. AbdelGawad

Answer

- Exercise on Slide 24.

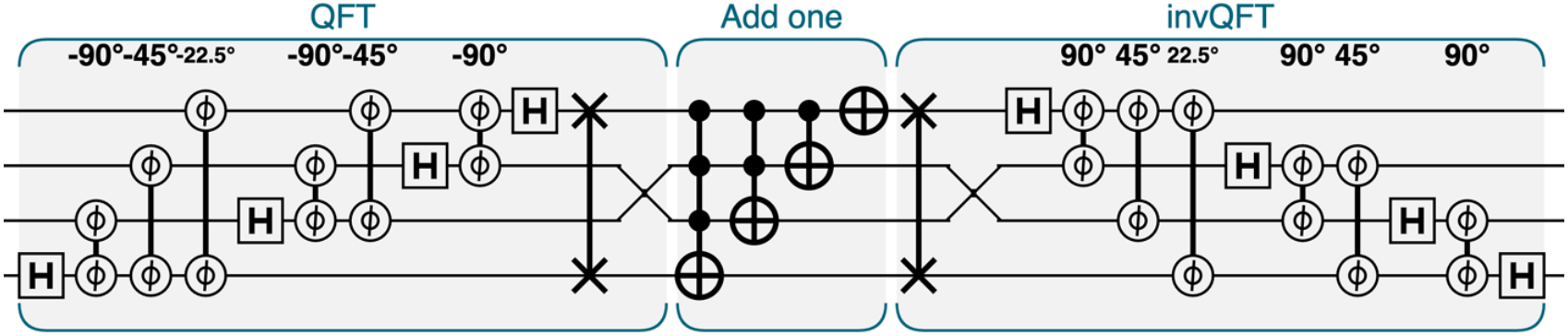
Answer: `45→180`.

- Let's test it.

Copyright © 2019-24 Moez A. AbdelGawad

Homework

- Explain quantum program (from Lect 0: Why QC).



  - Now we are ready … to read, understand, and explain each *single* step (of 28 steps).
    - Hint: Trace in QCEngine. Ignore green stuff (QFT, …) and what program as a whole does.
      - Reading Ch. 7 is unnecessary, and in fact not quite helpful now.
  - Written homework (plain English, by email. No code).

Copyright © 2019-24 Moez A. AbdelGawad

Course Webpage

http://eng.staff.alexu.edu.eg/staff/moez/teaching/pqc-f19

(download lecture slides)

Copyright © 2019-24 Moez A. AbdelGawad

Next Lecture Appetizer!

- In next lecture:
  - Move on to applications.
  - Build libraries, modules, and toolboxes, using primitives presented earlier.
    - Useful as parts in bigger and more advanced applications.
  - Quantum Arithmetic and Logic.
  - Amplitude Amplification.
  - (Introduce) Quantum Fourier Transform (QFT).

Copyright © 2019-24 Moez A. AbdelGawad

Thank You

Copyright © 2019-24 Moez A. AbdelGawad